A high-reliability, non-CRP-discard arbiter PUF based on delay difference quantization

A High-Reliability, Non-CRP-Discard Arbiter PUF Based on Delay Difference Quantization

Yao Wang, *Member, IEEE*, Guangyang Zhang, Xue Mei, Chongyan Gu, *Senior Member, IEEE*

***Abstract*—As a lightweight hardware security primitive, physical unclonable functions (PUFs) can provide reliable identity authentication for the Internet of Things (IoT) devices with limited resources. Arbiter PUF (APUF) is one of the most well-known PUF circuits. However, its hardware implementation has poor reliability on field programmable gate arrays (FPGAs). This paper proposed a highly reliable APUF that uses a delay difference quantization strategy (DDQ-APUF). By adding multiple configurable delay units to the two symmetrical paths of the conventional APUF, the delay difference between the two symmetrical paths of APUF can be obtained by collecting the output of APUF under different delay configurations. Compared to conventional APUFs, DDQ-APUF does not use the arbitration result of signal transmission in two symmetric paths as its response, but rather uses the quantified delay difference between the two paths as its response. A tolerance threshold is adopted in the authentication to accommodate the variations in delay differences due to environmental changes. Moreover, the modeling attack resistance of DDQ-APUF is evaluated, and a strategy for improving this resistance by incorporating pseudo-XOR technique is proposed. The circuit was implemented on Xilinx Artix-7 FPGAs and the experimental results show that the reliability achieves 99.95% with non-CRP-discard.**

***Index Terms*—DDQ-APUF, FPGA, IoTs, Security, Authentication.**

I. Introduction

THE progress of the Internet of Things (IoTs), with billions of connected devices, offers assistance to our regular routines. With the expansion of the scale of IoTs, information security has become a major challenge for the sustainable development of IoTs. Currently, the security of traditional encryption algorithms, including Advanced Encryption Standard (AES), Data Encryption Algorithm (DES) and Ron Rivest Adi Shamirh Len Adlema (RSA), relies on the confidentiality of the secret key, which means that users can authenticate and decrypt the device in possession of the secret key. However, the secret key is commonly stored in a non-volatile memory (NVM). The secret key can be easily obtained by attackers through physical means, such as invasive attacks and side-channel attacks. To address this, physical unclonable functions (PUFs), as a type of hardware security module that can effectively resist physical attacks with low overhead, have been widely developed and applied for identification, authentication, and key generation [1]. PUF is a hardware security primitive that utilizes the unpredictable changes that occur during silicon chip manufacturing to produce unique challenge-response pairs (CRPs), like a chip's digital fingerprint. Even the same manufacturer cannot clone two chips with the same CRPs. These CRPs are not stored in the non-volatile memory (NVM) but are generated on each activation, avoiding the problem of keys stored in the NVM that can be obtained by attackers through physical attacks [2].

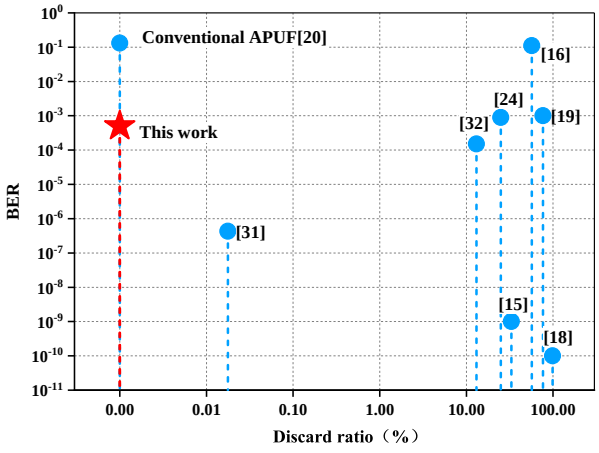


Fig. 1. BER and CRP discard ratio of state-of-the-art strong PUFs.

Up to now, researchers have proposed a variety of PUF circuits, including delay-based PUFs (such as arbiter PUF (APUF) [3]), memory-based PUFs (such as the SRAM-PUF [4]), oscillator-based PUFs (Ring Oscillator PUF (RO-PUF) [5]), etc. As a member of the strong PUFs, APUF has a large number of CRPs and is often used for secure authentication [6]. Moreover, APUF has many advantages, such as low hardware overhead and low power consumption. APUF is one of the most explored PUF circuits and has been used as the bottom block in many complex PUF architectures such as feedforward arbiter PUF [7], XOR PUF [8], and lightweight PUF [9]. However, due to the metastability of the delayed flip-flop (DFF) arbiter and the routing constraints of FPGA, conventional APUF has poor reliability in FPGA implementations [10]. Furthermore, previous studies have shown that, for some designs, more than a quarter of the responses are susceptible to environmental changes (voltage, temperature, and aging) [11] [12]. The unreliability causes

This work was partially supported by the Science and Technology Project of Henan Province (232102211076), and partially supported by the EPSRC New Investigator Award (EP/X009602/1) and Innovate UK SCHEME Project (IUK10065634).*(Corresponding author: Yao Wang, Chongyan Gu.)*

Yao Wang, Guangyang Zhang and Xue Mei are with the School of Electrical and Information Engineering, Zhengzhou University, Zhengzhou, 450001, China (e-mail: ieyaowang@zzu.edu.cn)

Chongyan Gu is with the Centre for Secure Information Technologies (CSIT), Queen's University Belfast, Belfast BT3 9DT, U.K. (e-mail: C.Gu@qub.ac.uk)

Manuscript received xx xx, xxxx; revised xx xx, xxxx.

some raw responses registered during authentication cannot be accurately reproduced. Hence, improvement of APUF reliability is an important prerequisite for PUF applications. In previous studies, helper data algorithms (HDA) were proposed to extract reliable keys from unstable responses [13] [14]. HDA typically employs error-correcting codes, which require significant hardware overhead and storage space. For strong PUFs, the storage space required for error-correcting codes to protect the massive amount of CRPs is astonishing, making this approach only suitable for weak PUFs. Additionally, error-correcting codes may leak sensitive information about CRPs to attackers, posing a security threat. Strong PUFs often use CRP filtering to enhance reliability [15] [16]. Unfortunately, this approach will reduce the challenge-response space, which may be exploited to accelerate modeling attacks [17], and result in a less secure PUF-based authentication scheme.

In this paper, we introduce a new type of APUF, the Delay Difference Quantization APUF (DDQ-APUF), which is not only highly reliable but also does not require the discarding of any CRPs. Fig. 1 shows the discard ratios of CRPs and the Bit Error Rate (BER) for various APUF structures. It is noteworthy that recent studies typically improve system reliability by discarding a large number of unstable CRPs. For example, [18] retained only 1% of CRPs to achieve 100% reliability, [15] and [19] discarded 33.15% and 76.43%, respectively, to reach nearly 100% reliability. While DDQ-APUF achieved 99.95% reliability without discarding any CRPs.

The main contributions of this work are summarized as follows:

1) We propose DDQ-APUF by adding multiple configurable delay units to two symmetrical signal transmission paths of the conventional APUF. The output data of DDQ-APUF under different delay configurations as its response, contains information on the delay difference of the two symmetric paths. Essentially, the output response of conventional APUFs only reflects which of the two symmetric paths has a smaller delay, whereas the output response of DDQ-APUF includes specific quantitative information on the delay difference between the two paths. Therefore, the output response of DDQ-APUF contains richer information on the unique process variations of every single chip.
2) We set a threshold for the quantitative delay difference in the authentication phase. The responses, whose corresponding delay differences fall within the tolerance range, can pass authentication. This method can accommodate the variation of delay difference due to environmental changes and improve the reliability of PUF. The output response will be obfuscated by a linear feedback shift register (LFSR), increasing the uniformity of the response.
3) The DDQ-APUF is implemented on a Xilinx Artix-7 FPGA board. The test results show that the proposed DDQ-APUF achieves very high reliability without CRP filtering. The hardware resource requirement of the DDQ-APUF is lower than those of the state-of-the-art mechanisms, and it is suitable for resource-constrained systems.

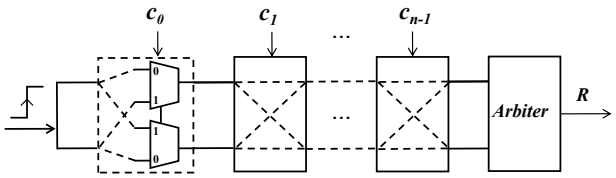


Fig. 2. The structure diagram of n-stage APUF.

The rest of this paper is organized as follows. Section II describes the background of this study. Section III demonstrates the details of the proposed DDQ-APUF. Section IV discusses the experimental results of DDQ-APUF. Finally, Section V summarizes this paper.

II. Background

A. Conventional APUF

The basic APUF structure consists of two symmetrical signal transmission paths with several delay stages and an arbiter [3], as shown in Fig.1. Each stage consists of a pair of multiplexers configured as a crossbar switch, which is controlled by a challenge c_i. It generates a response by comparing the time it takes the same rising edge pulse to reach the final arbiter through two symmetric circuits. Due to the process deviation in the fabrication of the circuit, the two symmetrical circuits are slightly different, so the rising edge signals in the two paths will not reach the arbiter at the same time. Therefore APUF is classified as a delay-based PUF. The arbiter output R is equal to 1 when the upper path signal passes faster, otherwise R is equal to 0. For an n-stage APUF, it can generate 2^n CRPs.

The 1-bit response of APUF only shows which path is faster, but cannot indicate the quantitative delay difference between the two paths. Unfortunately, the delay of the circuit will be affected by environmental changes (voltage, temperature, and aging), resulting in the unreliability of some original responses with small delay difference, as shown in Fig. 3(a). When the delay difference between the two paths is large enough as shown in Fig. 3(b), the value of the delay difference may change under environmental variations, but the polarity of the delay difference does not flip, hence the response is stable.

As illustrated in Fig. 3(c), the response of a conventional APUF can be expressed as

$$Response = \begin{cases} 1 & \Delta D > 0 \\ 0 & \Delta D < 0 \end{cases} \quad (1)$$

where $\Delta D = delay_{bottom} - delay_{top}$, whose polarity determines the response of APUF. When ΔD is close to 0, the response is sensitive to environmental changes. Note that the value of ΔD is changing continuously under environmental variation. If we can obtain the quantitative delay difference ΔD as the response and set an appropriate threshold in the authentication phase, the reliability of APUF can be improved dramatically.

One of the main challenges in implementing APUF on FPGAs is signal path asymmetry caused by layout and routing

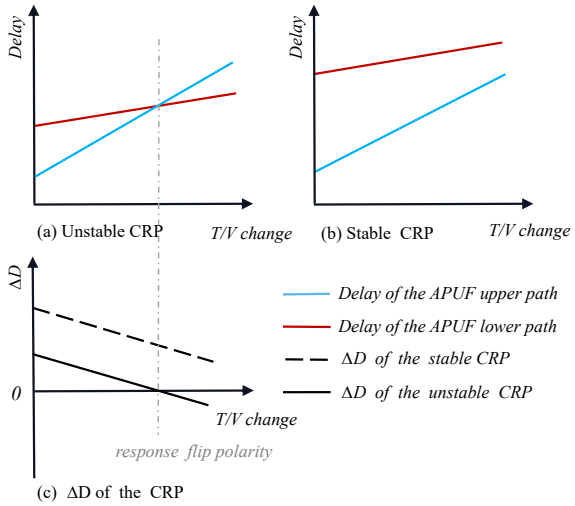


Fig. 3. The delay varies with temperature/voltage.

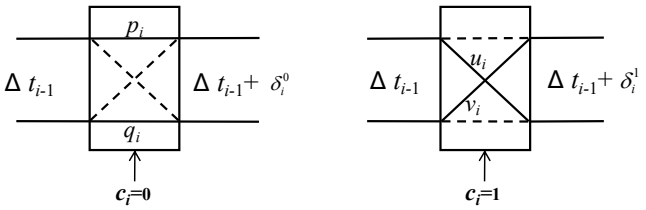


Fig. 4. Two possible path selections for a particular stage i

limitations. Previous works have used Programmable Delay Lines (PDL) to improve delay element precision and address this issue [20], [21], [22]. In this work, we not only employ programmable delay units to mitigate the path asymmetry but also introduce additional delay units to quantify the delay difference between the two paths, expanding the output response from 1-bit to 13-bit.

B. Mathematical Modeling of APUF

In the previous section, we provided an overview of the basic APUF. In the following section, we will briefly introduce the mathematical model of APUF [23]. For the traditional APUF, we know that the challenge bit c_i determines the specific propagation path in stage i. The four paths of stage i are defined as q, p, u, and v. When c_i = 0, the path of the i stage is parallel, and the delay difference between the two symmetric circuits is defined as δ_i^0; When c_i = 1, the path of the i stage is cross, and the delay difference between the two symmetric circuits is defined as δ_i^1. δ_i^0 and δ_i^1 can be expressed as

$$\begin{aligned} \delta_i^0 &= p_i - q_i \\ \delta_i^1 &= u_i - v_i \end{aligned} \tag{2}$$

Let Δt_i be the sum of the delay differences up to stage i. Referring to Fig 4. We can obtain the following equation.

$$\Delta t_i = (1 - 2c_i) \cdot \Delta t_{i-1} + (1 - c_i) \cdot \delta_i^0 + c_i \cdot \delta_i^1 \tag{3}$$

Accordingly, we can express the final delay difference Δt_{n-1} as

$$\Delta t_{n-1} = \Omega \cdot \Phi(C) \tag{4}$$

where the delay vector is Ω=(ω_0,ω_1,...,ω_n), the parameters of which are given by

$$\begin{aligned} \omega_0 &= \frac{\delta_0^0 - \delta_0^1}{2} \\ \omega_n &= \frac{\delta_{n-1}^0 + \delta_{n-1}^1}{2} \\ \omega_i &= \frac{\delta_{i-1}^0 + \delta_{i-1}^1 + \delta_i^0 - \delta_i^1}{2} \end{aligned} \tag{5}$$

where the feature vector $\Phi(C)$ represents the feature vector extracted from the n-bit input challenge, $\Phi(C)$=($\phi_1(C)$, $\phi_2(C)$,...,$\phi_k(C)$,1)^T,with $\phi_i(C)$=$\prod_{j=i}^{k}(1-2c_j)$.

The response of the APUF can be further expressed as

$$r = \varepsilon(\Delta t_{n-1}) \tag{6}$$

where ε denotes the Heaviside step function, i.e.,$\varepsilon(\Delta t_{n-1}) = 0$ if $\Delta t_{n-1} < 0$ and $\varepsilon(\Delta t_{n-1}) = 1$ if $\Delta t_{n-1} > 0$.

C. Related Work of Reliability Enhancement Techniques

Two main methods are commonly applied to improve the reliability of PUFs. The first method is postprocessing, such as majority voting [24] and error correction codes (ECC) [25]. The majority voting can improve reliability to a certain extent, but some unstable CRPs may still remain. ECC can achieve extremely high reliability, but it requires storing helper data. As the number of CRPs increases, the storage space required for the helper data also grows. This makes it suitable only for weak PUFs with a small number of CRPs and inapplicable to strong PUFs that have a large number of CRPs [26]. For example, [27] proposed a method using the Bose-Chaudhuri-Hocquenghem (BCH) code for error correction. In this scheme, to obtain a 63-bit secure key, the system needs to generate 255 bits of the original data, of which 63 bits are the actual key bits and 192 bits are the parity bits used for error correction. Moreover, the auxiliary data stored in NVM may potentially leak confidential information to attackers and threaten the security of PUF [28]. Compared to ECC, Maximum likelihood decoding scheme can achieve the same reliability with a lower hardware overhead [29], but it has the disadvantage of requiring multiple instances of PUF for key generation. Therefore, many lightweight reliability enhancement techniques are proposed to minimize the Bit Error Rate (BER) of PUF before applying ECC to reduce the hardware overhead. In [30], a multiple-arbiter scheme has been proposed to detect arbiter metastability by inserting either 4-DFF or SR-latch arbiter after each pair of multiplexers. However, this technique leads to response imbalance and introduces additional hardware overhead for detecting metastable states. In [10], two trigger arbiters are used for metastability detection. This unique challenge-response quadruple classification greatly reduces the errors generated by the PUF and decreases the correction burden during postprocessing. However, this approach still faces the problem of large hardware overhead.

The second approach to fortify the PUF reliability is CRP filtering [31], [32]. This method involves repeatedly generating CRPs under different conditions (for example, varying temperatures or operating voltages) and selecting CRPs that can be reliably reproduced, while discarding those that cannot be consistently replicated across different environments. The method of repeated measurements generally requires considerable time and costs [15]. Many efficient methods have been introduced to achieve high reliability. They can be classified into two categories. The first involves an integration of self-test circuits within PUF circuits. Such methods often require an addition of extra circuits to identify unstable CRPs, increasing both the hardware resource consumption and complexity [33], [34], [35]. Another method employs machine learning models to predict PUF delay differences to filter out unreliable challenges, without the need of additional hardware resources [36], [37], [19]. For example, [37] uses machine learning to select stable CRPs, with a training set consisting of 5000 CRPs. Each CRP needs to be tested 100,000 times repeatedly, making the process highly time-consuming. The work in [19] proposed a more efficient online reliability evaluation (ORE) scheme evaluates each CRP under three different delay configurations, requiring only three tests per CRP to achieve the same reliability. However, it is still possible that potentially unreliable CRPs may not be discarded due to the imperfect preselection criteria. In order to reduce the likelihood that potentially unstable CRPs are not discarded, the discard ratio range is set as wide as possible, which means that a high percentage of CRPs are discarded. For example, the bit-self-test(BST) strategy is used to flag unreliable CRPs [15]. Ultimately, only 66.85% of the CRPs are retained and the BER is reduced to 10^{-9}. By implementing CRP filtering, the PUF-FSM [18] selects only 1% of the total set of CRPs, resulting in a final reliability at 100%. This results in a significant reduction in the number of available CRPs. Moreover, during the CRP filtering process, the CRP reliability flag, which is the data used to mark reliable CRPs, is stored in NVM. This could potentially leak confidential information about the PUF to attackers, which will increase the modeling attack vulnerability and reduce the number of CRPs required for a successful modeling attack. [38], [39]. Therefore, it is highly desirable to have an approach that simultaneously achieves low overhead, minimal discarding of CRPs, and high reliability.

III. Proposed Method

A. Basic Concept

This section elucidates the fundamental principles of the DDQ-APUF. Initially, the objective is to ascertain the delay difference between two symmetrical paths. This is achieved by analyzing the output response of a conventional APUF structure, which reveals the path with lesser delay. Into this path, a configurable delay is introduced and incrementally increased until the output response of the APUF inverts. The magnitude of delay introduced at this juncture represents the delay difference between the symmetrical paths. Subsequently, this quantified delay difference, as opposed to the traditional APUF's delay difference polarity, serves as the PUF's response.

Fluctuations in environmental conditions, such as changes in temperature and operational voltage, may induce variability within a specific range in the DDQ-APUF's delay difference. During authentication, establishing a predefined tolerance margin allows authentic PUFs to be verified under the impact of environmental variations. Given that the authentication protocol incorporates a substantial number of bits in the PUF response, encompassing delay difference from several DDQ-APUF units, an illegitimate PUF's single unit delay difference might coincidentally fall within the authentication tolerance. However, the collective responses from multiple PUF units are improbable to entirely conform to this tolerance window. Thus, this approach not only guarantees the authentication of legitimate PUFs amidst environmental changes but also prevents the unauthorized verification of illegitimate PUFs. The setup of a tolerance threshold enhances the reliability of APUFs, but it may also introduce a misjudgment. Therefore, in the authentication phase, after balancing the reliability and misjudgment probability, the tolerance threshold is set at 3, with a detailed analysis provided below. Additionally, in the process of the threshold selection, the factors, such as process variations, temperature, and voltage changes, have been considered in the reliability test. As a result, the threshold we set is fixed and does not vary under different conditions.

B. DDQ-APUF Design

In this paper, 6 configurable delay units are added to the upper and lower paths of APUF respectively, as shown in Fig. 5. The delay unit has two states. The delay in the low latency state can be ignored, and the delay time in the high latency state is $delay_t$. Note that the number of configurable delay units on the shorter path must be sufficient to ensure that the arbiter's output can flip when all elements are set to high delay. The results of testing 10 PUF instances with 10,000 CRPs show that adding six extra delay units is sufficient. Therefore, 6 configurable delay units are added to the upper and lower paths of APUF respectively. We configure the delay units according to the phases $S_0 \sim S_{12}$, as shown in Fig. 5. In phases $S_0 \sim S_5$, all delay units on the lower path remain in a low latency state, while on the upper path, the number of delay units configured for high latency gradually decreases from six to only one. For S_0 phase, all the 6 delay units at upper path are set to high latency state, while all the delay units at lower path are configured to low latency state, and $R[0]$ is generated. For S_1 phase, 5 delay units at upper path are set to high latency state, while all the delay units at lower path are configured to low latency state, and $R[1]$ is generated. The rest can be done in the same manner. During phases S_7 to S_{12}, all delay units in the upper path are kept in a low latency state, while on the lower path, the number of delay units configured for high latency gradually increases from one to six. Note that $R[6]$ is generated during S_6 phase when all the 12 delay units are in low delay state. Ultimately, $R[0] \sim R[12]$ are generated in phases S_0 to S_{12}, respectively, and together form the 13-bit data R_O, which is taken as the output response of the DDQ-APUF. The approximation of ΔD can be obtained from R_O. Assuming R_O = 0000011111111, it can be seen that the

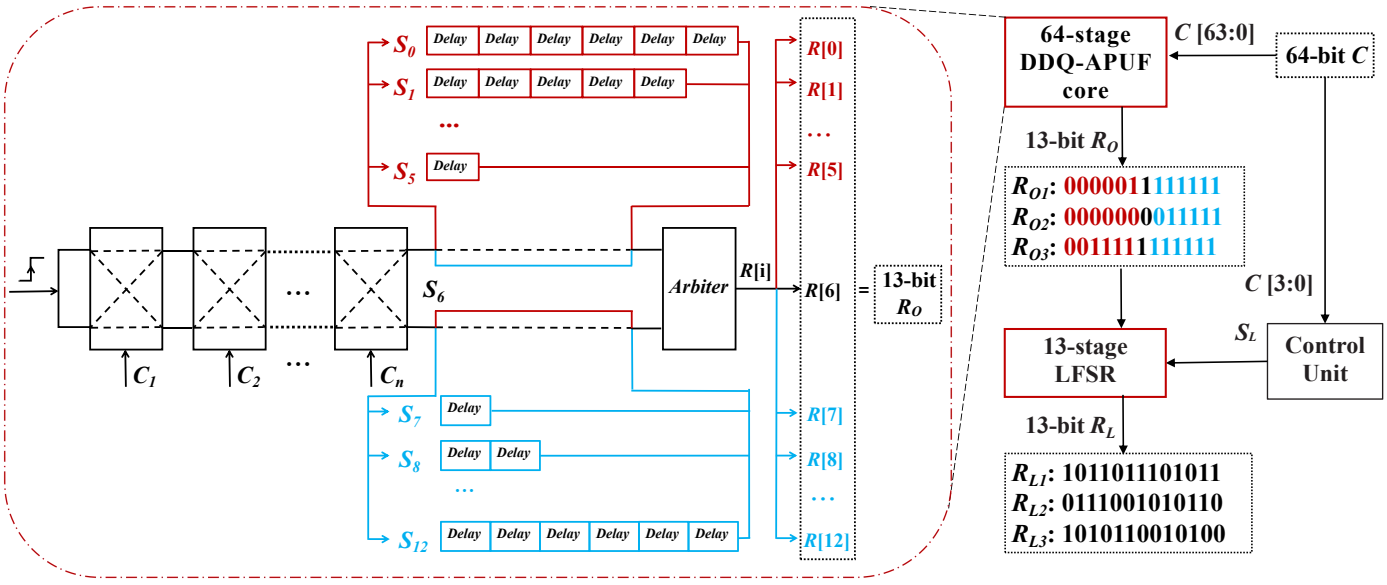


Fig. 5. Block diagram of the proposed DDQ-APUF.

response bits flip at $R[4]$ and $R[5]$. The value of ΔD in S_4 and S_5 phases are given by

$$\Delta D_4 = delay_{bottom} - delay_{top} - 2 * delay_t < 0 \quad (7)$$

$$\Delta D_5 = delay_{bottom} - delay_{top} - delay_t > 0 \quad (8)$$

where ΔD_4 and ΔD_5 represent the difference in delay between the lower and upper paths of DDQ-APUF in phases S_4 and S_5, respectively. $delay_{bottom}$ and $delay_{top}$ are the inherent delay times of the lower and upper paths of the APUF, respectively, excluding any additional configurable delay units. Therefore, we can deduce that the inherent delay difference ΔD between the lower and upper paths of the APUF lies between $delay_t$ and $2{*}delay_t$. In this way, the delay difference quantization is achieved.

Fig. 6 illustrates the difference in the probability distribution of the output responses between traditional APUF and DDQ-APUF, as well as during the authentication process. As shown by the horizontal axis in Fig. 6(b), we use the number of '1's in R_O to indicate the transition point from 0 to 1. The conventional APUF has a 1-bit response, while the DDQ-APUF has a 13-bit response with 14 types of value. The conventional APUF must generate a response equal to the original one to pass authentication. In contrast, ΔD of the DDQ-APUF may vary slightly under environmental changes, but it can still pass authentication as long as the response variation is within a threshold. Therefore, DDQ-APUF is fault-tolerant and highly reliable. For example, the deep red mark at 7 in Fig. 6(b) indicates that there are seven bits of 1 in R_O. It can be inferred that a transition occurs between $R[6]$ and $R[7]$, thus deducing that $\Delta D{<}delay_t$. Due to the small path delay difference at this point, it is sensitive to environmental noise, causing the output of the arbiter to flip, which would result in the failure of traditional APUF authentication. DDQ-APUF adopts a fault tolerance threshold as shown by the light red color in Fig.5(b). If the noise-induced flipped response bit falls within the fault tolerance threshold, the response can still be authenticated successfully.

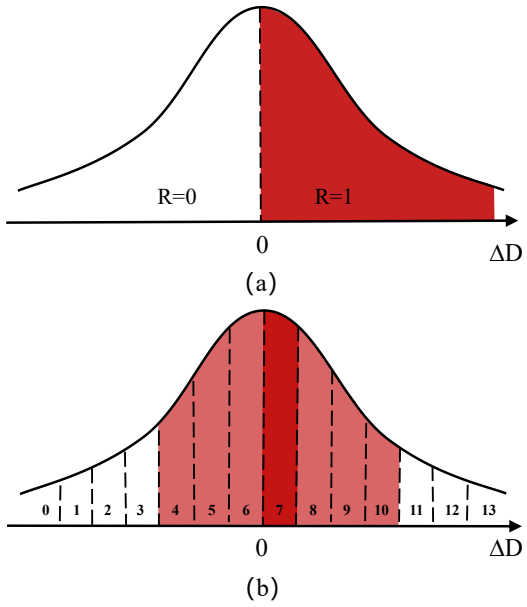


Fig. 6. Probability distribution of the number of '1' in the response of (a) conventional APUF and (b) DDQ-APUF.

However, the responses generated by DDQ-APUF are not uniform but comply with normal distribution, as shown in Fig. 6(b). To improve the uniformity of DDQ-APUF, we obfuscate the 13-bit response R_O by using it as the seed of an LFSR.

The number of shifts S_L of the LFSR is determined by 4-bit challenge data. The final output of the LFSR is used as the final response of the DDQ-APUF. In this way, the uniformity of DDQ-APUF is improved. Fig. 5 shows an example that 3 responses $R_{O1} \sim R_{O3}$ are obfuscated by the LFSR to generate 3 final responses $R_{L1} \sim R_{L3}$. During the authentication phase, the response R_O can be easily recovered with the same type LFSR, R_L and the same S_L.

C. Mathematical Model of Delay Difference Quantization Method

According to the mathematical model of APUF and the mathematical principle of the DDQ-APUF structure, the model of delay difference quantization is described as

$$R[i] = \varepsilon(\Delta t_{n-1} + (i-6) \cdot delay_t) \tag{9}$$

where $R[i]$ refers to the response output at each phase from S_0 to S_{12}. The 13-bit response R_O before LFSR obfuscating can be expressed as

$$R_O = (R[0], R[1], R[2], ..., R[12]) \tag{10}$$

Given that the delay differences for the two symmetrical signal transmission paths are typically minimal, we suggest that incorporating up to six delay units on one path should suffice to trigger an inversion in the arbiter's output.

D. Theoretical Analysis of Delay Difference Quantization Method

While the challenge-response behavior of a PUF can and actually often is modeled mathematically, PUFs are ultimately physical objects. It thus makes sense to describe our framework through formal language.

A PUF P is a physical system that can be stimulated with so-called challenges C from a challenge set $C_p \subseteq (0,1)^k$, upon which it reacts by producing corresponding responses r_i from a response set $R_p \subseteq (0,1)^m$ [40]. We denote this process as $r \leftarrow P(C)$. Since PUF P is a physical device, it suffers from measurement noise. This means that repeating the same input challenge to P can result in different response bits. we define the failure probability p_b of PUF P with respect to a challenge $C \subseteq C_p$ as

$$p_b = Pr(r \neq r', r \leftarrow P(C), r' \leftarrow P(C)) \tag{11}$$

where $r, r' \in R_p$ and the probability is over measurement noise. From the legitimate user perspective, we define the reliability of P with respect to C as 1-p_b. Let p_b be the probability that the response bit flips due to measurement noise, given a selected challenge C.

Algorithm 1 Authentication response.

```
procedure Authentication response
    Define threshold=3
    R_O ← P(C), R'_O ← P(C)
    if R_O ≠ R'_O
        e = R_O ⊕ R'_O
        Δ_e = The number of 1 in e
        if (Δ_e ≤ threshold)
            printf True
        else
            printf False
    else
        printf True
end procedure
```

In Algorithm 1, we present a method to improve reliability. For the number of bits that differ between the regenerated response and the original response data, we have set a tolerance. R_O is the first measurement, R'_O a second measurement, and $e = R_O \oplus R'_O$ (XOR operation) represents the difference between the two responses, then Δ_e is the number of '1's in e. Due to measurement noise $R_O \neq R'_O$ for the same selected C, if Δ_e is within the fault tolerance threshold, authentication can still be passed; otherwise, it cannot pass the authentication. In this way, the reliability is improved.

It should be noted that by establishing a fault tolerance threshold, the conditions for authentication approval are broadened. This expansion can enhance the system's probability of correctly accepting legitimate inputs. However, it may also inadvertently increase the likelihood of erroneous acceptances, which might lead to instances of misjudgment. This phenomenon can be described as

$$p_w = Pr(\Delta_e \leq threshold, r_y \leftarrow P_y(C), r_n \leftarrow P_n(C)) \tag{12}$$

where p_w is the probability of a misjudgment, which indicates the likelihood that the responses r_y and r_n generated by a legitimate device P_y and an illegal device P_n are both within the authentication threshold for the same C. The nominal delay difference Δt at the arbiter, with respect to the set of all challenges, is assumed to be normally distributed [41]. This distribution has zero mean and standard deviation σ_0, as shown in Fig.7(a). In the context of Delay Difference Quantization, the distribution of traditional APUF delay differences can be transformed into a distribution of the transition points of R_O. This distribution follows a normal distribution with a mean of 7 and a standard deviation of $\sqrt{14}$, spanning a range from 1 to 13, as shown in Fig.7(b). The probability of flip point x is computed by integrating the probability density function (PDF).

$$\begin{aligned} Pr(X \leq x) &= \frac{1}{2} erfc\left(-\frac{x-\mu}{\sqrt{2}\sigma}\right) \\ \text{with } erfc(t) &= \frac{2}{\sqrt{\pi}} \int_t^{\infty} e^{-z^2} dz \end{aligned} \tag{13}$$

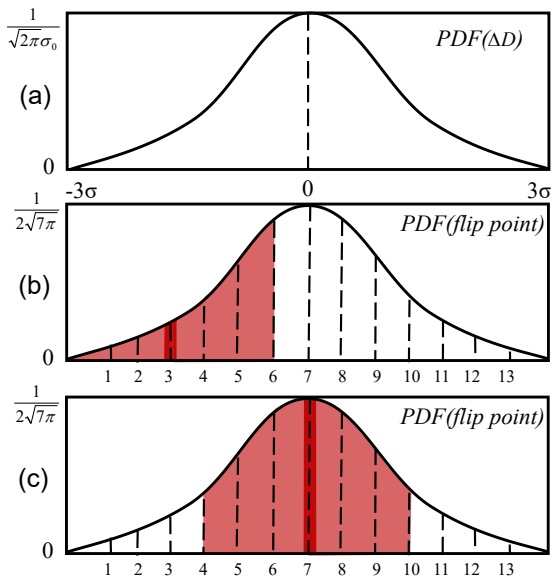


Fig. 7. (a) Probability density function curve of APUF response (b) Probability distribution of the flip point at 3 (c)Probability distribution of the flip point at midpoint 7.

where μ is the mean of the normal distribution and σ is the standard deviation. Therefore, the misjudgment probability formula can be expressed as

$$p_w = Pr(x + threshold) - Pr(x - threshold) \qquad (14)$$

It can be seen from Fig.7(b) and (c) that the p_w are generally different for different flip points. when the flip point is at the midpoint 7, the probability of misjudgment is maximized, as shown in Fig.7(c). Consequently, setting the threshold at 7 can result in a potential maximum error rate of 58.2%. Fortunately, a single device usually contains multiple PUFs, generating multiple responses under one challenge. A device is only validated if all responses are successfully authenticated; otherwise, it's marked as illegitimate. Thus, the likelihood of misidentifying an illegitimate device as legitimate diminishes as the number of PUFs involved in authentication increases. Taking the authentication of 64 PUFs embedded in single device as an example, and assuming that the p_w for each authentication is at its maximum, the cumulative p_w would be $(58.2\%)^{64}$, evidently much less than 1. Consequently, the DDQ-APUF achieves high reliability with negligible misjudgment probability p_w.

E. Resistance to Modeling Attack

Ideally, the output response of an APUF is only determined by internal variations and thus unpredictable to attackers. However, due to the absence of mechanisms to restrict access to CRPs, attackers can easily construct mathematical models through machine learning (ML) after collecting a significant number of CRPs, thereby accurately simulating and predicting the behavior of the target APUF [42]. Additionally, there is an approximation attack method that simplifies complex logic operations into linear functions by analyzing the internal logic structure of PUFs, thereby approximating the mapping relationships [43]. Methods to enhance the resistance of PUFs to modeling attacks can be divided into two categories: challenge/response obfuscation [18], [44], [45], [46] and strengthening the nonlinearity of PUF circuits [7], [47], [48]. Adopting challenge or response obfuscation techniques to prevent attackers from obtaining the true CRPs of a PUF can effectively defend against black-box attacks. However, in white-box attacks, attackers obtain the structure of the PUF circuit through means such as reverse engineering, hence challenge or response obfuscation techniques cannot provide defense. Structural nonlinearization implements nonlinear PUF structures to obstruct ML-based attacks. Among the nonlinear PUF structures, XOR APUF [5] is one of the most extensively studied architectures. It executes the XOR operation across the response bits of multiple parallel APUFs, where theoretically, the complexity of modeling attacks is exponentially related to the number of parallel APUFs [49]. Inspired by XOR APUF, we propose pseudo-XOR (PXOR) DDQ-APUF to enhance its resistance against modeling attacks.

In this study, we evaluated the machine learning resistance of the structure proposed in this paper using four well-known attack algorithms, including Logistic Regression (LR), Support Vector Machine (SVM), Covariance Matrix Adaptation Evolution Strategy (CMA-ES), and Artificial Neural Networks (ANN). In the test, 80% of the dataset was used for training the machine learning models, with the remaining 20% used for testing. Fig. 9 shows the modeling attack prediction rates for the 64-stage APUF, DDQ-APUF, 3XOR APUF, and 3PXOR DDQ-APUF with different training set size. It can be intuitively seen from the figure that for APUF, all four ML algorithms achieve high prediction rates with a small dataset. However, among the other structures, only CMA-ES demonstrated an upward trend in prediction rates as the size of the training set is increased. Therefore, our discussion primarily focuses on the CMA-ES algorithm, a method proven to be effective and commonly utilized in attacking XOR-APUFs [50], [51], [52].

We assume that an adversary can obtain the circuit structure through reverse engineering and acquire a portion of the original CRPs through untrusted channels. Based on the method presented in [23], which utilizes the CMA-ES algorithm for modeling attacks on target PUFs, we have conducted attacks on several PUF structures proposed in this paper. Firstly, a Python model is constructed based on the target PUF to generate training and testing datasets. Subsequently, an adaptive function is developed according to the mathematical model of the target PUF. We select CMA-ES for the capability of using a customized fitness function, feed the solution from the fitness function back to CMA-ES, and continuously optimize the parameter vector obtained from CMA-ES until the desired parameter vector is achieved. Finally, the testing set and the optimal parameter vector obtained are used in the fitness function to determine the final prediction rate.

DDQ-APUF is a variant of APUF, hence susceptible to ML attacks. The output response of a DDQ-APUF before undergoing LFSR obfuscating is 13 bits, with 14 possible

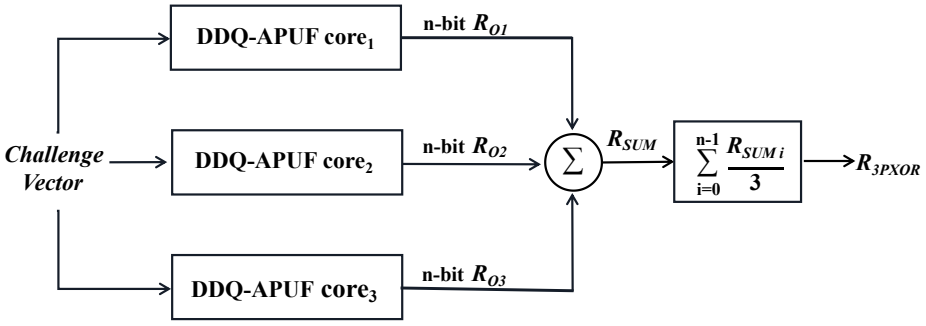


Fig. 8. Structure of 3PXOR DDQ-APUF.

values. We set a tolerance threshold (d =3) at the response flipping points, making the probability of passing the authentication by random guessing is 1/2, which is equivalent to that of APUF. However, when conducting mathematical modeling attacks on DDQ-APUF within a Python environment, without considering the influence of environmental noise and tolerance threshold, there are 14 possible outcomes for DDQ-APUF so that the probability of success through random guessing is 1/14, which is lower than the probability of passing authentication by random guess for APUF. Therefore, intuitively, setting a smaller tolerance can reduce the attacker's success rate of prediction, making DDQ-APUF more resistant to modeling attacks than conventional APUF.

The dashed lines in Fig. 9 demonstrate the prediction rates of utilizing CMA-ES to attack a conventional APUF, a DDQ-APUF without tolerance and a DDQ-APUF with a tolerance of 3. To achieve a 90% prediction rate, the traditional APUF only requires a training set of 500 CRPs, while the DDQ-APUF with a tolerance of 3 and the DDQ-APUF without any tolerance require training sets of 10,000 and 130,000 CRPs, respectively. Therefore, this experiment verifies that DDQ-APUF has stronger resistance to modeling attacks compared to traditional APUF, especially when a smaller tolerance is adopted.

To enhance the modeling attack resistance of DDQ-APUF, we introduce pseudo-XOR operations into DDQ-APUF to increase its nonlinearity. Fig. 8 shows the structure of a 3PXOR DDQ-APUF. $R_{O1}{\sim}R_{O3}$ represents the n-bit output responses generated by three DDQ-APUF cores with the same $ChallengeVector$. First, add the corresponding bits of $R_{O1}{\sim}R_{O3}$ to obtain a decimal sequence R_{SUM}. Then, sum the elements of R_{SUM} and divide by 3 to get R_{PXOR}. R_{SUM} and R_{PXOR} can be expressed as

$$R_{SUM} = R_{O1} + R_{O2} + R_{O3} \tag{15}$$

$$R_{PXOR} = \sum_{i=0}^{n-1} \frac{R_{SUM_i}}{3} \tag{16}$$

where R_{SUM_i} represents the i_{th} number in sequence R_{SUM}. Similar to DDQ-APUF, in the authentication process for 3PXOR DDQ-APUF, a tolerance threshold for R_{PXOR} can be set to improve reliability. Although PXOR is a linear operation, it ensures that any change in the response output of any DDQ-APUF cores in the system will affect the final output of the system, theoretically improving the system's resilience to modeling attacks.

We evaluate the modeling attack resistance of 3XOR APUF and 3PXOR DDQ-APUF under different tolerances using CMA-ES algorithm, with the results shown as solid lines in Fig. 9. The 3XOR APUF achieves a 90% prediction rate with 20,000 CRPs, while the 3PXOR DDQ-APUF requires 25,000 CRPs at a tolerance threshold $d = 3$, and 27,000 CRPs at $d = 2$. When training with 2 million CRPs, a prediction rate of 86% is reached at $d = 1$, and only 48% at $d = 0$. These experimental results indicate that 3PXOR DDQ-APUF has stronger resistance to ML attacks compared to 3XOR-APUF, especially when selecting lower tolerance, which can further significantly enhance its capability to resist ML attacks.

In addition to changing the circuit structure by increasing the number of PUF primitives as mentioned above, we can also enhance the security of PUFs through various protocols. For instance, by utilizing the erasable PUF technique [53], access to each CRP can be controlled, making it impossible to remeasure an erased CRP. An active deception protocol can also be used to prevent ML-based modeling attacks [54]. The protocol outputs real CRPs at certain time intervals and inserts a large number of fake CRPs between the correct CRPs. Therefore, the adversary will have to either wait for an impractically long time to collect enough real CRPs by directly querying the device or the ML model derived from the collected CRPs will be poisoned. All the above-mentioned anti-attack techniques can be utilized to enhance the ML attack resistance of DDQ-APUF.

IV. Experimental Results

We implemented a 64-bit DDQ-APUF on Xilinx Artix-7 XC7A100T FPGAs. All the configurable delay units in the DDQ-APUF core are implemented by using LUTs [20]. As shown in Fig. 10, the internal structure of an LUT is composed of some SRAMs and several MUXs. When a series of signals (A_1, A_2, A_3, A_4, A_5 and A_6) are entered, a particular SRAM cell will be selected and wired directly to the output of the LUT. An n-input LUT can be configured to implement any n-input combinational logic function. By changing the input signals of the LUT, it is possible to alter its internal signal transmission paths, thereby achieving control over its transmission delay.

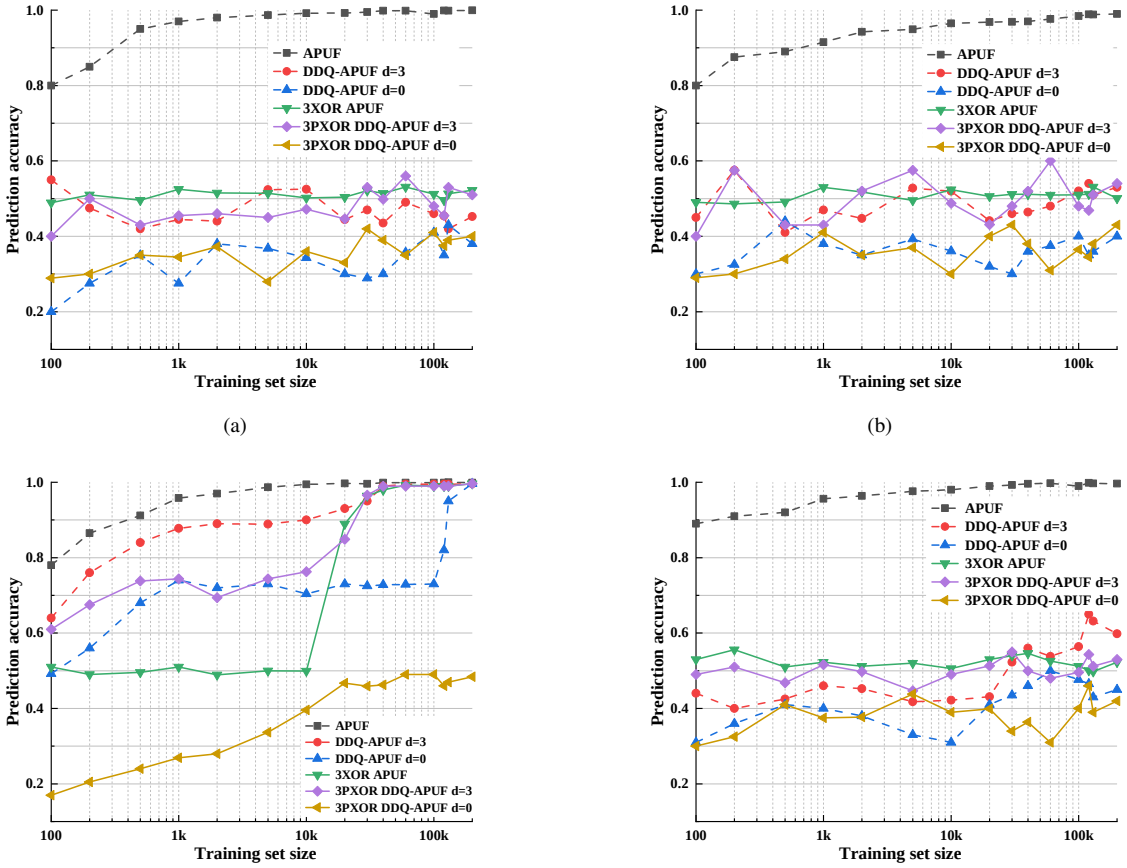


Fig. 9. Resistance of APUF, DDQ-APUF, 3XOR APUF and 3PXOR DDQ-APUF Under four ML algorithms(i.e., LR, SVM, CMA-ES and ANN). (a) Resistance against LR attacks. (b) Resistance against SVM attacks. (c) Resistance against CMA-ES attacks. (d) Resistance against ANN attacks. (Note: The random guess probability of APUF and 3XOR APUF is 1/2, while for DDQ-APUF and 3PXOR DDQ-APUF, the probability of random guessing is $(2d+1)/14$.

To alleviate the asymmetry issue in the implementation of APUF on FPGA, two PDLs are utilized to form a non-path-swapping switch block, which alleviates system delay deviations between the two delay chains [20], [21]. Additionally, to make the delays of the two paths as close as possible, an automatic fine-tuning strategy [22] is employed to eliminate path asymmetry and delay skew caused by system variations. Specifically, the system performs multiple measurements of the PUF output and automatically adjusts the PDL settings based on the measurement results until the uniformity of the PUF output approaches 50%.

The red and blue arrows in Fig. 10 represent the signal transmission paths with maximum and minimum delays. For example, if $A_2A_3A_4A_5A_6$ = '11111', the signal propagates through the red-line path, whereas if $A_2A_3A_4A_5A_6$ = '00000', the signal propagates through the path marked with the blue-lines. The lower blue-line path is slightly longer than the upper red-line path which results in a large propagation delay difference between these two paths. As shown in Fig. 11(a), when the upper path is connected to a LUT whose delay configuration signals $A_2A_3A_4A_5A_6$ are '00000', and the lower path is connected to a LUT whose delay configuration signals $A_2A_3A_4A_5A_6$ are '11111', the delay of the upper path

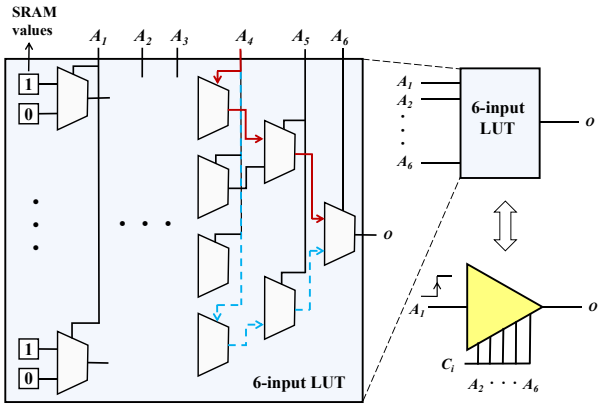


Fig. 10. The internal structure of a 6-input LUT.

is increased, which is equivalent to adding a delay to the upper path. Conversely, it is equivalent to adding a delay to the lower path.

A computer is used as the host to send challenges and receive responses from the FPGA. Next, we present the performance of the proposed DDQ-APUF derived from experimental

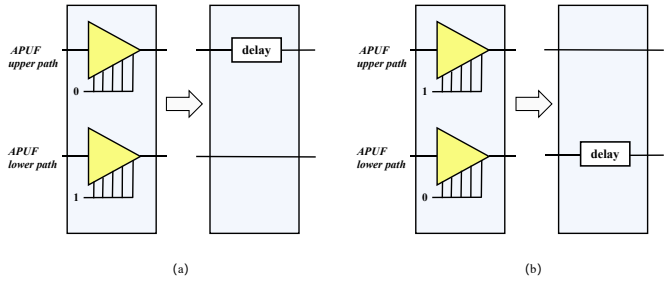


Fig. 11. The structure of delay unit.

tests. The concept of Hamming Distance (HD) will be used in this section:

$$HD(X,Y) = \frac{1}{m}\left(\sum_{i=0}^{m-1} X[i] \oplus Y[i]\right) \times 100\% \qquad (17)$$

where X and Y are two m-bit streams and $X[i]$ represents the i-th bit of X.

A. Uniformity

The uniformity metric is used as an estimate of the fraction of '1' in a response string. The probability of occurrence of '1' in the 13-bit PUF responses is measured, the ideal value is 50%. Since the DDQ-APUF response is multi-bit, the uniformity is defined as

$$Uniformity(R_i) = \left(\frac{1}{n}\sum_{j=1}^{n} R_j[i]\right) \times 100\% \qquad (18)$$

where i is the i-th bit of the response and n represents the number of responses, R_j is the j-th response under different challenges. In our experiment, 10,000 challenges are randomly generated and input into three chips with 10 DDQ-APUF instances per chip. The uniformity of R_O is not ideal because R_O always starts with '0' and ends with '1', so the uniformity gradually increases with the number of bits in R_O. To address this issue, R_O is feeded into an LFSR and generate R_L with an uniformity of around 50%, as shown in Fig. 12.

B. Uniqueness

Uniqueness is defined as if the same challenge is given, the degree of difference that should exist between the responses generated by different PUF instances for the same PUF design. Ideally, the uniqueness should be 50%. The uniqueness of a PUF can be obtained by measuring its inter-chip HD. The definition of uniqueness is given by

$$Uniqueness = \left(1 - \frac{2}{k(k-1)}\sum_{x=1}^{k-1}\sum_{y=i+1}^{k}\frac{HD(R_x,R_y)}{n}\right) \times 100\% \qquad (19)$$

where k represents the number of PUF instances tested, R_x and R_y respectively represent the responses generated by the x-th and y-th instances under the same test conditions, and n represents the number of all responses of PUF instances.

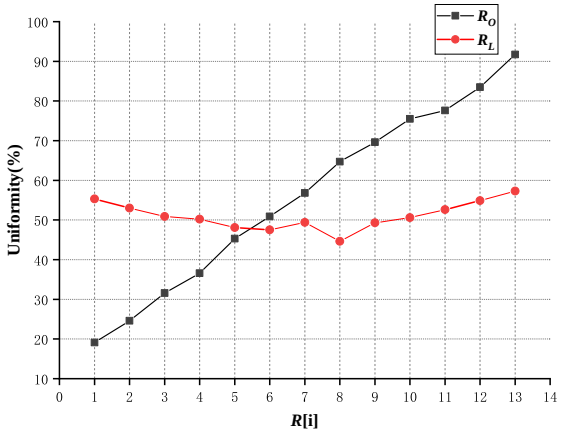


Fig. 12. The uniformity of DDQ-APUF.

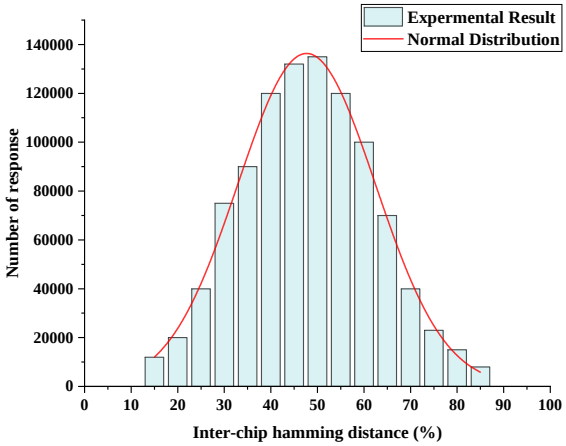


Fig. 13. The uniqueness of DDQ-APUF.

We input 10,000 randomly generated challenges into three chips with 10 DDQ-APUF instances per chip, and the average uniqueness measured is 47.28%, as shown in Fig. 13.

C. Reliability

Reliability is typically used to assess the reproducibility of a PUF's response to the same challenge in different environments, and ideally, PUF reliability is 100%. This can be obtained by measuring the on-chip HD of the PUF. The definition of reliability is given by

$$Reliability = \left(1 - \frac{1}{k}\sum_{i=1}^{k}\sum_{j=1}^{n}\frac{S_{ij}}{n}\right) \times 100\% \qquad (20)$$

$$S_{ij} = \begin{cases} 1 & HD(R_j,R_{ij}) > d \\ 0 & HD(R_j,R_{ij}) < d \end{cases} \qquad (21)$$

where k is the number of tests, n is the number of CRPs, R_j is the original response for the j-th challenge, R_{ij} is the R_j under the i-th test, S_{ij} represents whether R_{ij} is an error code, and d is the threshold for fault tolerance in authentication. Fig. 14 presents the number of '1' in the responses of the

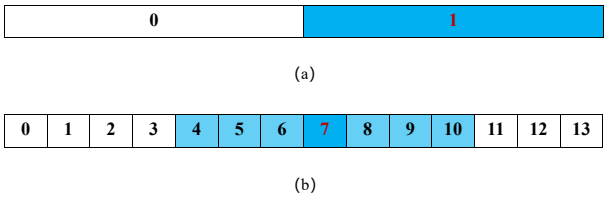


Fig. 14. The number of '1' in the responses of (a) the conventional APUF and (b) DDQ-APUF in the authentication phase.

conventional APUF and DDQ-APUF. Assuming the original responses of the conventional APUF and DDQ-APUF are '1' and '0000001111111', respectively. Therefore, the original number in Fig. 14 (a) and (b) are '1' and '7', respectively. Note that the latter can provide fault tolerance as highlighted by blue, but the former cannot.

In our experiments, 10,000 challenges were applied on 10 PUFs and the tests were repeated 30 times. The environmental temperature range is from -40°C to 80°C. The reliability is directly related to the threshold d. Note that the range of fault tolerance cannot exceed half of the total number of bits, because the authentication threshold of the conventional APUF is also half of the full range. The measured reliability is shown as the solid lines in Fig. 15. As can be seen, with the increase of the tolerance threshold d, the reliability of DDQ-APUF gradually improves, and when $d = 3$, the average reliability reaches 99.95%. Therefore, we set d to 3 finally.

Notably, applying XOR to multiple PUF instances can enhance resistance to modeling attacks, but it may also reduce the reliability [51], [55]. This is because if the response of a single PUF is affected by environmental noise and exhibits instability, XOR operations may accumulate these instabilities, thereby increasing the probability of erroneous responses in the final output. The 3PXOR DDQ-APUF can effectively resolve this issue. The DDQ-APUF outputs response data with quantified delay differences. The pseudo-XOR operation of the 3PXOR DDQ-APUF averages the delay difference data from three DDQ-APUFs. While the environmental noise might cause an increase or decrease to the delay difference data of a single DDQ-APUF, but the average delay difference of the three DDQ-APUFs is likely to remain close to the typical value. During the authentication phase, a tolerance threshold similar to that of the DDQ-APUF is set. Even if a single DDQ-APUF becomes unstable, it is still likely to pass authentication. Therefore, the proposed 3PXOR DDQ-APUF has a less affection by the pseudo-XOR operation compared with the XOR APUF.

D. Aging Experiment

We carried out the aging process by heating the FPGA at high temperatures. The FPGA board was operated at 80°C for 3 hours. Then the board was withdrawn from the stress environment and was cooled down for an hour. After that, the reliability of PUF was measured under different temperatures. We calculated the reliability using (7), and the results of the aging test are shown as the dashed lines in Fig. 15. When the

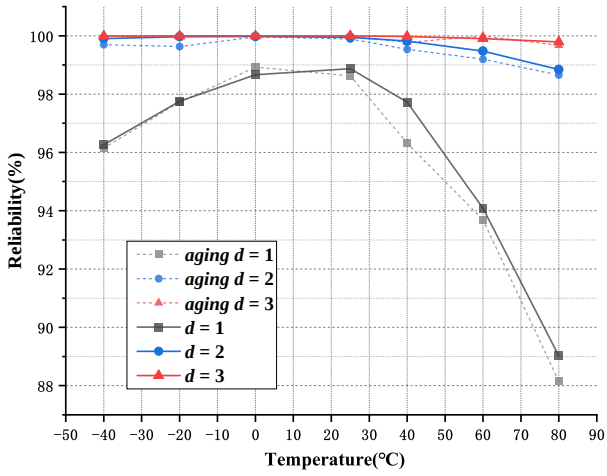


Fig. 15. Reliability of DDQ-APUF with different tolerances.

tolerance threshold $d = 1$, it is clear that the reliability reduces significantly compared with pre-aging conditions. However, with the increase of d, the effect of aging was mitigated. Therefore, the results verify that DDQ-APUF has aging resistance.

E. Performance Comparison

Table I presents a comparison between the DDQ-APUF, traditional APUF, and other improved PUFs reported in the literature. All of these PUF architectures are implemented on FPGAs, predominantly using the Artix-7 platform. Traditional APUF [20] demonstrates challenges in achieving high reliability on FPGA platforms, recording a reliability of merely 94%. FF-APUF [56], a compact improved PUF with low hardware overhead, attains a reliability of 97.1% without employing additional reliability enhancement techniques. [58] achieves 99.55% reliability using majority voting technique, but at a high hardware consumption. [57], [59], and [60] implement CRP filtering to achieve reliability ranging from 95% to 98.31%, yet [59] and [60] incur substantial hardware consumption. [10] and [23] achieve more than 99% reliability. However, [10] involves both CRP filtering and majority voting techniques, resulting in high hardware expenditure. [15] utilizes CRP filtering to achieve a low BER of 10^{-9} but discards 33.15% of CRPs. Notably, all improved PUFs, except for [56] and [58], require NVM to store the flag bits for marking unstable CRPs, potentially exposing confidential PUF information and increasing vulnerability to modeling attacks. In contrast, DDQ-APUF demonstrates superior reliability on FPGA platforms without discarding any CRPs or requiring NVM. Moreover, it exhibits excellent randomness and uniqueness characteristics. Regarding hardware resource utilization, DDQ-APUF also shows efficient usage, presenting lower hardware overhead compared to other improved PUFs.

V. Conclusions

This paper presented an APUF structure based on quantized delay differences, where the response data contains information on the delay differences of symmetric paths in the APUF.

TABLE I
PERFORMANCE COMPARISON WITH STATE-OF-THE-ART STRONG PUFS

	Type	Stage	Uniformity	Uniqueness	Reliability	Temperature Range	Technology	NVM Requirement	Hardware Consumption
WIFS'10 [20]	Conventional APUF	64	51.4%	51.6%	94.0%	$-40 \sim 80°C$	Artix-7	No	156 LUTs 69 FFs
		128	48.6%	51.3%	94.4%				302 LUTs 132 FFs
TIFS'19 [10]	MISR PUF	128	-	49.7%	100%[a]	$-40 \sim 90°C$	Artix-7	Yes	419 LUTs 264 FFs
ACCESS'20 [15]	BST-APUF	64	50.3%	49.1%	$<10^{-9}$(BER)[b]	$-30 \sim 80°C$	Artix-7	Yes	150 LUTs 47 FFs
TETC'21 [56]	FF-APUF	64	47%	41.53%	97.10%	$0 \sim 75°C$	Artix-7	No	384 LUTs 512 FFs
TCASI'21 [57]	RSO-APUF	128	-	49.8%	95%[b]	-	Artix-7	Yes	395 LUTs 176 FFs 4 KB NVM
ACCESS'22 [58]	APUF	64	51.84%	46.21%	99.55%[c]	$0 \sim 85°C$	Artix-7	No	87 slices[d]
IOTJ'22 [59]	CT PUF	64	50%	50%	95.87% [b]	$-50 \sim 125°C$	Artix-7	Yes	731 LUTs 486 FFs
TIFS'23 [60]	TP PUF	64	50%	49.34%	98.31%[b]	-	ZYNQ-7000 SoC	Yes	1175 LUTs 451 FFs
IOTJ'23 [23]	(64,4)-OIPUF	64	50%	40%	99.0%[b]	$-20 \sim 100°C$	Artix-7	Yes	537 LUTs 8 FFs
This work	**DDQ-APUF**	64	50%	47.28%	99.95%	$-40 \sim 80°C$	Artix-7	No	237 LUTs 180 FFs
		128	50%	47.65%	99.91%				365 LUTs 216 FFs

[a] *With CRP filtering and majority voting.*
[b] *With CRP filtering.*
[c] *With majority voting.*
[d] *Each slice contains 4 LUTs, 8 FFs and other logical resources.*

By setting a tolerance for the delay difference during the authentication phase, it achieves a reliability of over 99.9% at temperatures ranging from -40 to 80 °C, without discarding CRPs, the requirement for NVM, and with low hardware overhead. Additionally, the DDQ-APUF has stronger resistance to ML attacks than traditional APUF. The introduction of pseudo-XOR in DDQ-APUF can further enhance its resistance to ML attacks, surpassing that of XOR-APUF. The proposed DDQ-APUF is suitable for security authentication in IoTs devices.

REFERENCES

[1] B. Gassend, D. Clarke, M. Van Dijk, and S. Devadas, "Silicon physical random functions," in *Proc. ACM Conf. Computer Commun. Secur. (CCS)*, 2002, pp. 148–160.

[2] D. Lim, J. W. Lee, B. Gassend, G. E. Suh, M. Van Dijk, and S. Devadas, "Extracting secret keys from integrated circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 13, no. 10, pp. 1200–1205, 2005.

[3] J. W. Lee, D. Lim, B. Gassend, G. E. Suh, M. Van Dijk, and S. Devadas, "A technique to build a secret key in integrated circuits for identification and authentication applications," in *IEEE Symp VLSI Circuits Dig Tech Pap*, 2004, pp. 176–179.

[4] J. Guajardo, S. S. Kumar, G.-J. Schrijen, and P. Tuyls, "FPGA intrinsic PUFs and their use for IP protection," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst. (CHES)*, 2007, pp. 63–80.

[5] G. E. Suh and S. Devadas, "Physical unclonable functions for device authentication and secret key generation," in *Proc. Des. Autom. Conf. (DAC)*, 2007, pp. 9–14.

[6] S. Hemavathy and V. K. Bhaaskaran, "Arbiter PUF-a review of design, composition, and security aspects," *IEEE Access*, 2023.

[7] B. Gassend, D. Lim, D. Clarke, M. Van Dijk, and S. Devadas, "Identification and authentication of integrated circuits," *Concurrency and Computation: Practice and Experience*, vol. 16, no. 11, pp. 1077–1098, 2004.

[8] A. Maiti and P. Schaumont, "Improved ring oscillator PUF: An FPGA-friendly secure primitive," *J. Cryptology*, vol. 24, pp. 375–397, 2011.

[9] M. Majzoobi, F. Koushanfar, and M. Potkonjak, "Lightweight secure PUFs," in *IEEE ACM Int. Conf. Comput. Des. Dig. Tech. Pap. (ICCAD)*, 2008, pp. 670–673.

[10] S. S. Zalivaka, A. A. Ivaniuk, and C.-H. Chang, "Reliable and modeling attack resistant authentication of arbiter PUF in FPGA implementation with trinary quadruple response," *IEEE Trans. Inf. Forensic Secur.*, vol. 14, no. 4, pp. 1109–1123, 2018.

[11] M. Bhargava, C. Cakir, and K. Mai, "Comparison of bi-stable and delay-based physical unclonable functions from measurements in 65nm bulk CMOS," in *Proc. Custom. Integr. Circuits Conf. (CICC)*, 2012, pp. 1–4.

[12] R. Maes, V. Rozic, I. Verbauwhede, P. Koeberl, E. Van der Sluis, and V. van der Leest, "Experimental evaluation of physically unclonable functions in 65 nm CMOS," in *European Solid-State Circuits Conf.*, 2012, pp. 486–489.

[13] J. Delvaux, D. Gu, D. Schellekens, and I. Verbauwhede, "Helper data algorithms for PUF-based key generation: Overview and analysis," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 34, no. 6, pp. 889–902, 2014.

[14] A. A. Pour, F. Afghah, D. Hély, V. Beroulle, G. Di Natale, A. R. Korenda, and B. Cambou, "Helper data masking for physically unclonable function-based key generation algorithms," *IEEE Access*, vol. 10, pp. 40 150–40 164, 2022.

[15] Z. He, W. Chen, L. Zhang, G. Chi, Q. Gao, and L. Harn, "A highly reliable arbiter PUF with improved uniqueness in FPGA implementation using bit-self-test," *IEEE Access*, vol. 8, pp. 181 751–181 762, 2020.

[16] C.-C. Lin and M.-S. Chen, "Enhancing reliability and security: A configurable poisoning PUF against modeling attacks," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4301–4312, 2022.

[17] A. Vijayakumar, V. C. Patil, C. B. Prado, and S. Kundu, "Machine learning resistant strong PUF: Possible or a pipe dream?" in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust, (HOST)*, 2016, pp. 19–24.

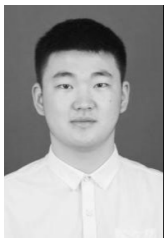
[18] Y. Gao, H. Ma, S. F. Al-Sarawi, D. Abbott, and D. C. Ranasinghe, "PUF-FSM: A controlled strong PUF," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 37, no. 5, pp. 1104–1108, 2018.

[19] C. Ma, J. Mu, J. Ye, S. Chen, Y. Cao, H. Li, and X. Li, "Online reliability evaluation design: Select reliable CRPs for arbiter PUF and its variants," in *Proc. European Test Workshop*, 2023, pp. 1–6.

[20] M. Majzoobi, F. Koushanfar, and S. Devadas, "FPGA PUF using

programmable delay lines," in *Proc. IEEE Int. Workshop Inf. Forensics Secur. (WIFS)*, 2010, pp. 1–6.
[21] D. P. Sahoo, R. S. Chakraborty, and D. Mukhopadhyay, "Towards ideal arbiter PUF design on Xilinx FPGA: A practitioner's perspective," in *Proc. Euromicro Conf. Digit. Syst. Des.*, 2015, pp. 559–562.
[22] M. Majzoobi, A. Kharaya, F. Koushanfar, and S. Devadas, "Automated design, implementation, and evaluation of arbiter-based PUF on FPGA using programmable delay lines," *Cryptol. ePrint Arch.*, 2014. [Online]. Available: https://eprint.iacr.org/2014/639
[23] C. Xu, L. Zhang, M.-K. Law, X. Zhao, P.-I. Mak, and R. P. Martins, "Modeling attack resistant strong PUF exploiting stagewise obfuscated interconnections with improved reliability," *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16 300–16 315, 2023.
[24] Y. Choi, B. Karpinskyy, K.-M. Ahn, Y. Kim, S. Kwon, J. Park, Y. Lee, and M. Noh, "Physically unclonable function in 28nm FDSOI technology achieving high reliability for AEC-Q100 grade 1 and ISO26262 ASIL-B," in *IEEE Int. Solid-State Circuits Conf.(ISSCC) Dig. Tech. Papers*, 2020, pp. 426–428.
[25] R. Maes, A. Van Herrewege, and I. Verbauwhede, "PUFKY: A fully functional PUF-based cryptographic key generator," in *Lect. Notes Comput. Sci.*, 2012, pp. 302–319.
[26] Y. Gao, H. Ma, G. Li, S. Zeitouni, S. F. Al-Sarawi, D. Abbott, A.-R. Sadeghi, and D. C. Ranasinghe, "Exploiting PUF models for error free response generation," *arXiv preprint arXiv:1701.08241*, 2017.
[27] G. E. Suh, C. W. O'Donnell, and S. Devadas, "Aegis: A single-chip secure processor," *IEEE Des. Test Comput.*, vol. 24, no. 6, pp. 570–580, 2007.
[28] D. Jeon, J. H. Baek, Y.-D. Kim, J. Lee, D. K. Kim, and B.-D. Choi, "A physical unclonable function with bit error rate $< 2.3 \times 10^{-8}$ based on contact formation probability without error correction code," *IEEE J. Solid-State Circuit*, vol. 55, no. 3, pp. 805–816, 2019.
[29] M.-D. Yu, M. Hiller, and S. Devadas, "Maximum-likelihood decoding of device-specific multi-bit symbols for reliable key generation," in *Proc. IEEE Int. Symp. Hardw.-Oriented Secur. Trust (HOST)*, 2015, pp. 38–43.
[30] S. S. Zalivaka, A. V. Puchkov, V. P. Klybik, A. A. Ivaniuk, and C.-H. Chang, "Multi-valued arbiters for quality enhancement of PUF responses on FPGA implementation," in *Proc. Asia South Pac. Des. Autom. Conf. (ASP-DAC)*, 2016, pp. 533–538.
[31] D. Jeon, D. Lee, D. K. Kim, and B.-D. Choi, "A $325F^2$ physical unclonable function based on contact failure probability with bit error rate $<$ 0.43 ppm after preselection with 0.0177% discard ratio," *IEEE J. Solid-State Circuit*, vol. 58, no. 4, pp. 1185–1196, 2022.
[32] K. Yang, Q. Dong, D. Blaauw, and D. Sylvester, "A $553F^2$ 2-transistor amplifier-based physically unclonable function (PUF) with 1.67
[33] M. Bhargava and K. Mai, "An efficient reliable PUF-based cryptographic key generator in 65nm CMOS," in *Proc. Des. Autom. Test Eur. (DATE)*, 2014, pp. 1–6.
[34] Y. Shifman, A. Miller, O. Keren, Y. Weizmann, and J. Shor, "A method to improve reliability in a 65-nm SRAM PUF array," *IEEE Solid-State Circ. Lett.*, vol. 1, no. 6, pp. 138–141, 2018.
[35] K. Liu, Y. Min, X. Yang, H. Sun, and H. Shinohara, "A $373F^2$ 2D power-gated EE SRAM physically unclonable function with dark-bit detection technique," in *IEEE Asian Solid-State Circuits Conf. (A-SSCC)*, 2018, pp. 161–164.
[36] X. Xu, W. Burleson, and D. E. Holcomb, "Using statistical models to improve the reliability of delay-based PUFs," in *Proc. IEEE Comput. Soc. Annu. Symp. on VLSI (ISVLSI)*, 2016, pp. 547–552.
[37] C. Zhou, K. K. Parhi, and C. H. Kim, "Secure and reliable XOR arbiter PUF design: An experimental study based on 1 trillion challenge response pair measurements," in *Proc. Des. Autom. Conf. (DAC)*, 2017, pp. 1–6.
[38] G. T. Becker, "On the pitfalls of using arbiter-PUFs as building blocks," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 34, no. 8, pp. 1295–1307, 2015.
[39] J. Delvaux, "Machine-learning attacks on PolyPUFs, OB-PUFs, RPUFs, LHS-PUFs, and PUF-FSMs," *IEEE Trans. Inf. Forensic Secur.*, vol. 14, no. 8, pp. 2043–2058, 2019.
[40] C. Jin, W. Burleson, M. van Dijk, and U. Rührmair, "Programmable access-controlled and generic erasable PUF design and its applications," *J. Cryptogr. Eng.*, vol. 12, no. 4, pp. 413–432, 2022.
[41] Y. Lao and K. K. Parhi, "Statistical analysis of MUX-based physical unclonable functions," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 33, no. 5, pp. 649–662, 2014.
[42] J. Liu, Y. Zhu, C.-H. Chan, and R. P. Martins, "An entropy-source-preselection-based strong PUF with strong resilience to machine learning attacks and high energy efficiency," *IEEE Trans. Circuits Syst. I-Regul. Pap.*, vol. 69, no. 12, pp. 5108–5120, 2022.
[43] J. Shi, Y. Lu, and J. Zhang, "Approximation attacks on strong PUFs," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2138–2151, 2020.
[44] Y. Wang, X. Mei, Z. Chang, W. Fan, B. Guo, Z. Quan, and D. K. Jain, "A lightweight authentication protocol against modeling attacks based on a novel LFSR-APUF," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 283–295, 2024.
[45] Z. Chang, S. Shi, B. Song, W. Fan, and Y. Wang, "Modeling attack resistant arbiter PUF with time-variant obfuscation scheme," in *Proc. -Int. Conf. Field-Program. Log. Appl. (FPL)*, 2021, pp. 60–63.
[46] S. Shi, Z. Chang, B. Guo, and Y. Wang, "Modeling attack resistant arbiter PUF based on dynamic finite field matrix multiplication scheme," in *Proc. EEE Int. Conf. Solid-State Integr. Circuit Technol. (ICSICT)*, 2022, pp. 1–3.
[47] D. P. Sahoo, D. Mukhopadhyay, R. S. Chakraborty, and P. H. Nguyen, "A multiplexer-based arbiter PUF composition with enhanced reliability and security," *IEEE Trans. Comput.*, vol. 67, no. 3, pp. 403–417, 2017.
[48] X. Xu and J. Zhang, "Rethinking FPGA security in the new era of artificial intelligence," in *Proc. - Int. Symp. Qual. Electron. Des. (ISQED)*, 2020, pp. 46–51.
[49] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber, "Modeling attacks on physical unclonable functions," in *Proc. ACM Conf. Computer Commun. Secur. (CCS)*, 2010, pp. 237–249.
[50] G. T. Becker, "The gap between promise and reality: On the insecurity of XOR arbiter PUFs," in *Lect. Notes Comput. Sci.*, 2015, pp. 535–555.
[51] N. Sayadi, P. H. Nguyen, M. van Dijk, and C. Jin, "Breaking XOR arbiter PUFs without reliability information," *arXiv preprint arXiv:2312.01256*, 2023.
[52] J. Zhang, C. Xu, M.-K. Law, Y. Jiang, X. Zhao, P.-I. Mak, and R. P. Martins, "A 4T/cell amplifier-chain-based XOR PUF with strong machine learning attack resilience," *IEEE Trans. Circuits Syst. I-Regul. Pap.*, vol. 69, no. 1, pp. 366–377, 2021.
[53] C. Jin, W. Burleson, M. van Dijk, and U. Rührmair, "Programmable access-controlled and generic erasable PUF design and its applications," *J. Cryptogr. Eng.*, vol. 12, no. 4, pp. 413–432, 2022.
[54] C. Gu, C.-H. Chang, W. Liu, S. Yu, Y. Wang, and M. O'Neill, "A modeling attack resistant deception technique for securing lightweight-PUF-based authentication," *IEEE Trans. Comput-Aided Des. Integr. Circuits Syst.*, vol. 40, no. 6, pp. 1183–1196, 2021.
[55] R. Naveenkumar, N. Sivamangai, A. Napolean, and S. S. S. Priya, "Design and evaluation of XOR arbiter physical unclonable function and its implementation on FPGA in hardware security applications," *J. Electron. Test.-Theory Appl.*, vol. 38, no. 6, pp. 653–666, 2022.
[56] C. Gu, W. Liu, Y. Cui, N. Hanley, M. O'Neill, and F. Lombardi, "A flip-flop based arbiter physical unclonable function (APUF) design with high entropy and uniqueness for FPGA implementation," *IEEE Trans. Emerg. Top. Comput.*, vol. 9, no. 4, pp. 1853–1866, 2021.
[57] J. Zhang and C. Shen, "Set-based obfuscation for strong PUFs against machine learning attacks," *IEEE Trans. Circuits Syst. I-Regul. Pap.*, vol. 68, no. 1, pp. 288–300, 2021.
[58] N. N. Anandakumar, M. S. Hashmi, and M. A. Chaudhary, "Implementation of efficient XOR arbiter PUF on FPGA with enhanced uniqueness and security," *IEEE Access*, vol. 10, pp. 129 832–129 842, 2022.
[59] J. Zhang, C. Shen, Z. Guo, Q. Wu, and W. Chang, "CT PUF: Configurable tristate PUF against machine learning attacks for IoT security," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 14 452–14 462, 2022.
[60] C. Xu, J. Zhang, M.-K. Law, X. Zhao, P.-I. Mak, and R. P. Martins, "Transfer-path-based hardware-reuse strong PUF achieving modeling attack resilience with 200 million training CRPs," *IEEE Trans. Inf. Forensic Secur.*, vol. 18, pp. 2188–2203, 2023.

Yao Wang (Member, IEEE) received his M.S. degree from Zhengzhou University, Zhengzhou, China, in 2009, and the Ph.D. degree from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2013. From 2014 to 2017, he was a Lecturer with UESTC, then he joined Zhengzhou University. Since 2018, he has been an Associate Professor at the School of Information Engineering, Zhengzhou University. His research interests include low-power mixed-signal integrated circuits and hardware security for IoT applications.



Guangyang Zhang was born in 1999 in Henan, China. He received a Bachelor of Science degree. He graduated from Nankai University Binhai College in China in 2021. He is currently pursuing the M.S. degree at Zhengzhou University in China. His current research interests include hardware security and digital integrated circuits.

Xue Mei was born in 1998 in Henan, China. She received a Bachelor of Science degree. She graduated from Henan Normal University in China in 2020. She is currently pursuing the M.S. degree at Zhengzhou University in China. Her current research interests include hardware security and digital integrated circuits.

Chongyan Gu (S'14–M'16–SM'21) received the Ph.D. degree from Queen's University Belfast, Belfast, U.K., in 2016. She is currently a Senior Lecturer (Associate Professor) in the School of EEECS at Queen's University Belfast. She has been awarded EPSRC New Investigator Award in 2022. Her research into physical unclonable function (PUF) has been utilised as part of a security architecture for electronic vehicle (EV) charging systems, licensed by LG-CNS, South Korea. Her team was the overall winner of INVENT 2015, a competition to accelerate the commercialisation of innovative ideas. She currently serves as an Associate Editor of IEEE Transactions on Emerging Technologies in Computing (TETC) and Guest Editor of IEEE Transactions on Circuits and Systems – I (TCAS-I). She also serves on the Organising Committee of AsianHOST 2024 and Technical Programme Committees of the ISCAS, ASP-DAC, and AsianHOST conferences. She served on the Organising Committee of AsianHOST 2023 and the workshop and tutorial chair of the 32nd FPL 2022 conference. She was invited to give keynote/tutorial talks to international conferences, such as, HiPEAC 2023 and IEEE ASP-DAC 2020. Her current research interests include PUFs, security in/for approximate computing, true random number generator (TRNGs), hardware Trojan detection and machine learning attacks. She is an IEEE Senior member.